

P3 ANSWERS

1) Discuss the race conditions of your solution, and where did you need to use locks to avoid them.

Every thread has access to the file in read only mode and each thread allocates its own histogram, meaning no race condition arises because each thread works on its own allocated memory. For this reason we didn't need to use any lock to avoid conflicts between threads.

2) In your solution, how many threads can simultaneously read from the file? And update the histogram?

Because each thread uses their own file descriptor, N number of threads can work at the same time. They also have their own histogram array therefore updating can be done by N threads at the same time.

3) Report the needed execution time as a function of the number of threads. Comment your results.

For comparison the sequential program had an execution time of: 0.120s

Threads (n)	Execution Time (T)	Observation
1	0.128s	On average it takes more than doing it sequential since it needs to create the thread
2	0.093s	It becomes faster because the work is split between threads
4	0.070s	The more workers there are, the less one worker has to read, thus being more efficient
8	0.069s	On average it is the same time as with 4
16+	0.075s	Results start to get worse since the time to create a thread and recompile all the data the results takes more as we add more threads